

Proyecto Final

El proyecto final estará dividido en **dos entregas principales**. La primera entrega será el **PSet #4**, enfocado en discovery, requerimientos, diseño inicial, planificación y configuración DevOps. La segunda entrega será el **Proyecto Final**, enfocado en la implementación completa del MVP.

Parte 1: PSet #4

Discovery, Requirements & DevOps Setup

1. Objetivo del PSet #4

En esta primera fase, cada equipo debe transformar una oportunidad de negocio ambigua en un proyecto de software bien definido. El objetivo no es construir toda la aplicación todavía, sino entregar una base profesional para la implementación.

Al finalizar el PSet #4, cada equipo debe tener:

- documentación de discovery,
 - requerimientos funcionales y no funcionales,
 - reglas de negocio,
 - roles y permisos,
 - alcance MVP,
 - UML,
 - repositorio GitHub estructurado,
 - GitHub Project con issues como historias de usuario,
 - GitHub Actions configurado,
 - ambientes **dev** y **prod** conectados a Render,
 - flujo CI/CD funcionando.
-

2. Dinámica cliente / equipo desarrollador

Cada equipo recibirá un caso de negocio asignado por el profesor. Los equipos trabajarán en parejas: un grupo actuará como **cliente** y otro como **equipo desarrollador**. Luego se intercambian los roles.

El grupo cliente debe responder como si representara a la empresa con el problema de negocio. No debe entregar todo perfectamente definido; debe actuar como un cliente real, con necesidades, prioridades, restricciones y ambigüedades.

El grupo desarrollador debe realizar discovery, hacer preguntas, aclarar dudas, identificar reglas, proponer alcance y documentar la solución.

3. Casos de negocio posibles

El profesor asignará uno de los siguientes casos:

AccessFlow — Solicitudes de acceso interno

Sistema para gestionar solicitudes de acceso a herramientas internas como GitHub, bases de datos, dashboards, CRM o ambientes cloud.

ExpenseFlow — Aprobación de gastos y reembolsos

Sistema para registrar, revisar, aprobar o rechazar solicitudes de gasto de empleados.

ChangeFlow — Solicitudes de cambios técnicos

Sistema para controlar cambios técnicos, configuraciones, integraciones, permisos críticos o despliegues manuales.

IncidentFlow — Escalamiento de incidentes operativos

Sistema para gestionar incidentes críticos, asignar responsables, registrar timeline, notificar y cerrar con resumen.

4. Documento de discovery

Cada equipo debe entregar un documento en:

`/docs/discovery.md`

El documento debe incluir:

Resumen ejecutivo

Debe explicar el problema de negocio, usuarios principales, dolor actual y objetivo del MVP.

Contexto del negocio

Debe describir cómo funciona hoy el proceso, qué herramientas se usan, qué pasos son manuales y dónde se pierde trazabilidad.

Usuarios y roles

Debe identificar los usuarios del sistema, sus responsabilidades, restricciones y permisos.

Requerimientos funcionales

Deben describir capacidades concretas del sistema. Ejemplo:

RF-01: El sistema debe permitir que un usuario autenticado cree una solicitud.

RF-02: El sistema debe permitir que un aprobador apruebe o rechace una solicitud.

RF-03: El sistema debe enviar una notificación cuando una solicitud cambie de estado.

Requerimientos no funcionales

Deben considerar seguridad, trazabilidad, mantenibilidad, usabilidad, observabilidad, separación de ambientes y despliegue.

Reglas de negocio

Deben documentar condiciones importantes del caso.

Ejemplos:

- Gastos mayores a cierto monto requieren aprobación adicional,
- Accesos críticos requieren revisión de seguridad,
- Cambios de alto riesgo requieren plan de rollback,
- Incidentes críticos notifican inmediatamente.

Estados del flujo principal

Deben definir el ciclo de vida de la entidad principal, explicar cada estado y documentar transiciones válidas e inválidas.

Eventos del sistema

Deben identificar eventos relevantes, por ejemplo:

- REQUEST_CREATED
- REQUEST_SUBMITTED
- REQUEST_APPROVED
- REQUEST_REJECTED

- COMMENT_ADDED
- REQUEST_COMPLETED

Para cada evento deben indicar cuándo ocurre, quién lo dispara, qué datos contiene, si genera notificación y si debe registrarse en la auditoría.

Notificaciones

Deben definir qué eventos notifican, a quién, por qué canal y con qué prioridad.

Alcance MVP

Debe quedar claro qué entra en la primera versión.

Fuera de alcance

Debe quedar claro que no se construirá en esta versión.

Riesgos, supuestos y preguntas abiertas

Deben documentar riesgos técnicos, riesgos de alcance, supuestos y preguntas pendientes.

5. GitHub Project e issues

Cada equipo debe crear un **GitHub Project** con al menos estas columnas:

- Backlog
- Todo
- In Progress
- In Review
- Done

El trabajo debe organizarse como **historias de usuario**. No se aceptan issues genéricos como “hacer backend” o “hacer frontend” sin contexto.

Formato esperado:

Como [rol], quiero [acción], para [beneficio].

Cada integrante debe tener al menos **5 issues asignados**. Cada issue debe incluir:

- título claro,
- historia de usuario asociada,
- descripción,
- criterios de aceptación,

- label,
- assignee,
- prioridad.

Ejemplo:

Como Employee, quiero crear una solicitud de acceso para pedir permisos a una herramienta interna.

Criterios de aceptación:

- El usuario autenticado puede crear una solicitud.
 - La solicitud queda persistida.
 - El estado inicial es SUBMITTED.
 - Se publica el evento REQUEST_SUBMITTED.
-

6. Repositorio

El repositorio debe quedar estructurado, aunque la app todavía no esté implementada.

Estructura recomendada:

```
final-project/  
  backend/  
    app/  
    domain/  
    application/  
    infrastructure/  
    api/  
    tests/  
    requirements.txt  
    Dockerfile  
  frontend/  
    app/  
    Dockerfile  
  docs/  
    discovery.md  
    requirements.md  
    uml/  
  .github/  
    workflows/  
    ci-cd.yml  
  docker-compose.yml  
  README.md
```

La estructura puede variar, pero debe verse claramente la separación entre backend, frontend, documentación, infraestructura y workflows.

7. GitHub Actions y ambientes

Para el PSet #4 deben dejar configurada la base DevOps del proyecto.

El workflow debe cubrir dos ramas:

- `dev`
- `master` o `main`

La rama `dev` representa el ambiente de desarrollo.

La rama `master/main` representa producción.

CI en `dev`

Cuando haya pull request hacia `dev`, debe correr un workflow de CI que incluya:

- instalación de dependencias,
- lint,
- tests,
- build de imagen dev de Docker.

Cuando haya un merge a dev, debe correr un workflow de CI que incluya:

- correr validaciones,
- construir imagen dev Docker,
- enviarla a Docker Hub con tag dev,
- disparar redeploy en Render de dev.

CD hacia producción desde `master/main`

El despliegue a producción solo debe ocurrir cuando exista un push de tag a `master/main`.

Debe:

- correr validaciones,
- construir imagen prod Docker,
- enviarla a Docker Hub,
- disparar redeploy en Render de prod.

No se acepta despliegue directo a producción desde ramas feature.

8. Ambientes en Render

Deben configurar al menos dos ambientes:

```
dev → docker.io/<usuario>/<proyecto>:dev  
prod → docker.io/<usuario>/<proyecto>:prod
```

Deben guardar los deploy hooks de Render como secrets en GitHub:

```
DOCKERHUB_USERNAME  
DOCKERHUB_TOKEN  
RENDER_DEV_DEPLOY_HOOK  
RENDER_PROD_DEPLOY_HOOK
```

9. UML

Deben entregar diagramas en:

`/docs/uml/`

Use Case Diagram

Debe incluir actores principales y casos de uso del MVP.

Class Diagram inicial

Debe incluir entidades principales, atributos importantes y relaciones. Como mínimo:

- User
- Role
- Request / entidad principal del caso
- Approval / Decision
- Comment / Message, si aplica
- Notification
- AuditLog

También deben aparecer abstracciones importantes si ya están definidas:

- Repository interfaces
- State
- Event / Observer
- Command
- Strategy / Policy

Sequence Diagram

Debe modelar todos los flujos críticos:

Ejemplo:

Usuario crea solicitud → backend valida → persiste → publica evento → genera notificación → registra auditoría

10. README

El repositorio debe tener un `README.md` con:

- nombre del proyecto,
 - caso de negocio asignado,
 - integrantes,
 - descripción del problema,
 - stack técnico,
 - arquitectura propuesta,
 - cómo correr localmente,
 - cómo correr Docker Compose,
 - explicación del flujo GitHub Actions,
 - links a Render dev/prod si están disponibles.
-

Parte 2 — Proyecto Final

Implementación del MVP

1. Objetivo del Proyecto Final

En la segunda fase, cada equipo debe implementar el MVP definido en el PSet #4. La solución debe reflejar el discovery realizado, respetar el alcance acordado y demostrar dominio de arquitectura, patrones, backend, frontend, persistencia, DevOps y documentación.

El objetivo es construir una app **coherente, funcional, bien diseñada técnicamente**.

2. Alcance mínimo obligatorio

Cada equipo debe implementar una aplicación funcional que incluya:

- Patrones de diseño: creacionales y de comportamiento
 - autenticación,
 - usuarios con roles y permisos,
 - creación de la entidad principal del caso,
 - consulta por rol,
 - aprobación, asignación o decisión según el caso,
 - cambio de estado controlado,
 - notificaciones por eventos,
 - auditoría o historial,
 - frontend funcional,
 - backend con API,
 - persistencia real con ORM,
 - arquitectura hexagonal,
 - Docker Compose,
 - GitHub Actions,
 - despliegue en Render para **dev** y **prod**.
-

3. Stack

El proyecto final debe usar:

- **Backend:** FastAPI
 - **Frontend:** Streamlit
 - **Base de datos:** PostgreSQL
 - **ORM:** SQLAlchemy
 - **Infraestructura:** Docker Compose
 - **CI/CD:** GitHub Actions + Docker Hub + Render
 - **Arquitectura:** Hexagonal
-

4. Arquitectura esperada

La solución debe separar responsabilidades en capas.

Domain

Debe contener entidades, value objects, enums, reglas de negocio, interfaces, estados, eventos y abstracciones de patrones.

Application

Debe contener casos de uso, servicios de aplicación, DTOs, coordinación del flujo y lógica de orquestación.

Infrastructure

Debe contener modelos ORM, repositorios concretos, conexión a base de datos, providers externos, event bus concreto y configuración.

API

Debe contener rutas, dependencias, validación de acceso y controladores.

Frontend

Debe contener vistas, formularios, manejo de sesión/token y consumo de la API.

El dominio no debe depender del ORM, FastAPI, Streamlit ni servicios externos.

5. ORM y repositorios

El backend debe usar ORM en la capa de infraestructura. No se aceptan consultas ORM directamente en endpoints.

La separación esperada es:

- **DTOs:** entrada y salida de la API
- **Dominio:** entidades y reglas
- **ORM:** persistencia concreta
- **Repositorios:** interfaces + implementaciones concretas

Los casos de uso deben depender de abstracciones, no de detalles de infraestructura.

6. Patrones obligatorios

El proyecto final debe aplicar patrones de forma práctica, no solo nominal.

Patrones creacionales mínimos

Deben usar:

- Factory

- Abstract Factory or Builder
- Singleton, solo para registry/configuración, no para estado de negocio mutable

Opcional, solo si es necesario:

- Prototype

Patrones de comportamiento mínimos

Deben usar:

- State
- Observer
- Command
- Strategy
- Template Method

Los patrones deben estar justificados en el README y visibles en el diseño/código.

7. Funcionalidades mínimas del MVP

Aunque cada caso de negocio es distinto, todos los MVP deben cubrir este flujo:

Usuario autenticado → crea solicitud/registro → sistema valida → persiste → cambia estado → publica evento → notifica → registra auditoría/historial

Además, deben implementar:

- listado de registros según rol,
 - vista de detalle,
 - acción principal del flujo: aprobar, rechazar, asignar, escalar o completar,
 - notificaciones visibles para el usuario,
 - historial o auditoría de acciones importantes.
-

8. Frontend

El frontend debe permitir usar el sistema.

Debe incluir:

- login,
- listado principal,
- formulario de creación,

- detalle del registro,
- acciones según rol,
- vista de notificaciones,
- vista de historial o comentarios si aplica.

La autorización visual en frontend ayuda a la experiencia de usuario, pero la autorización real debe estar en backend.

9. Backend

El backend debe exponer endpoints suficientes para soportar el frontend.

10. DevOps

El proyecto debe mantener el flujo configurado en PSet #4 y probarlo en el desarrollo.

11. Entregables del Proyecto Final

Cada equipo debe entregar:

1. Repositorio completo.
 2. Código backend.
 3. Código frontend.
 4. Docker Compose funcional.
 5. Base de datos configurada.
 6. CI/CD funcional.
 7. App desplegada en dev y prod.
 8. README final.
 9. UML actualizado.
 10. Evidencia de GitHub Project.
 11. Demo funcional.
 12. Defensa técnica.
-

12. README final

El README debe incluir:

- descripción del problema,

- solución propuesta,
 - stack,
 - arquitectura,
 - patrones aplicados y justificación,
 - cómo correr localmente,
 - cómo correr con Docker Compose,
 - cómo funcionan los ambientes dev/prod,
 - links a despliegues,
 - usuarios de prueba,
 - flujo principal del sistema,
 - limitaciones conocidas.
-

13. Demo

Durante la demo, cada equipo debe mostrar:

- login,
 - flujo principal completo,
 - permisos por rol,
 - persistencia en base de datos,
 - cambio de estado,
 - evento publicado,
 - notificación generada,
 - auditoría o historial,
 - frontend funcional,
 - despliegue dev/prod,
 - pipeline GitHub Actions,
 - Docker Hub con imágenes publicadas.
-

14. Defensa individual

Cada estudiante podrá ser preguntado sobre cualquier parte del proyecto. La defensa puede incluir:

- explicar una decisión de diseño,
 - justificar un patrón,
 - explicar una parte del código,
 - modificar una validación,
 - agregar un endpoint pequeño,
 - corregir un bug simple,
 - explicar el flujo CI/CD,
 - leer un diagrama UML.
-

Rúbrica del PSet #4 — 100 pts

1. Discovery y requerimientos — 25 pts

Evalúa la claridad del problema, entendimiento del negocio, usuarios, roles, reglas de negocio, requerimientos funcionales/no funcionales, MVP, fuera de alcance, riesgos y preguntas abiertas.

2. GitHub Project e issues — 15 pts

Evalúa board organizado, historias de usuario reales, issues asignados por integrante, criterios de aceptación, labels, prioridades y trazabilidad del trabajo.

3. UML — 15 pts

Evalúa use case, class diagram y sequence diagram. Los diagramas deben ser coherentes con el caso de negocio y el discovery.

4. Estructura del repositorio — 10 pts

Evalúa separación clara entre backend, frontend, docs, infra, workflows y README.

5. GitHub Actions / CI — 15 pts

Evalúa workflow funcional en `dev`, instalación de dependencias, lint, tests y build de imagen Docker.

6. Docker Hub + Render / CD base — 15 pts

Evalúa imagen publicada, ambientes dev/prod configurados, secrets correctos y deploy hooks conectados.

7. README y presentación — 5 pts

Evalúa la claridad del README inicial y la capacidad del equipo para explicar el avance.

Rúbrica del Proyecto Final — 100 pts

1. Funcionalidad del MVP — 10 pts

Evalúa que el flujo principal esté completo, funcional y alineado al discovery. Debe incluir login, creación, consulta, acción principal, estados, notificaciones y auditoría/historial.

2. Arquitectura y separación de capas — 10 pts

Evalúa arquitectura hexagonal, alta cohesión, bajo acoplamiento, separación DTO/dominio/ORM y ausencia de lógica de negocio en endpoints.

3. Backend y persistencia — 10 pts

Evalúa FastAPI, endpoints, autenticación, autorización, ORM, repositorios, validaciones, reglas de negocio y persistencia real.

4. Frontend — 10 pts

Evalúa usabilidad, integración con API, login, vistas principales, formularios, acciones según rol y consulta de notificaciones/historial.

5. Patrones de diseño — 10 pts

Evalúa aplicación real y justificada de patrones creacionales y de comportamiento. No se aceptan patrones solo “de nombre”.

6. DevOps y despliegue — 10 pts

Evalúa Docker Compose, GitHub Actions, publicación en Docker Hub, despliegue dev/prod en Render y consistencia del flujo CI/CD.

7. UML y documentación — 5 pts

Evalúa diagramas actualizados, README final y documentación suficiente para correr y entender el sistema.

8. GitHub workflow — 5 pts

Evalúa el uso real del GitHub Project, issues, ramas, PRs, reviews y trazabilidad entre requerimientos e implementación.

9. Demo y defensa — 30 pts

Evalúa la claridad de la demo, el dominio técnico del equipo y la capacidad individual para explicar y modificar partes del sistema.

Recomendación

Este proyecto debe trabajarse como un ciclo profesional:

Discovery → Requerimientos → Diseño → Planificación → CI/CD base → Implementación → Despliegue
→ Demo → Defensa